

وأنا بتعلم C# اتعرفت على مفهوم مهم في التعامل مع الـ Strings وهو String Immutability.

يعني إيه Immutability ؟

ببساطة، الـ String في C# لا يمكن تعديل قيمته بعد إنشائه.

مثلاً:

```
string name = "Sara";
```

```
name = name + " Beshoi";
```

هنا الـ String الأصلي "Sara" لم يتم تعديله.

الـ .NET. أنشأ String جديد بقيمة "Sara Beshoi"، وبعدها أصبح الـ Variable name يشير إلى الـ String الجديد.

وده معناه إن:

```
string x = "Hello";
```

```
x = "Hi";
```

القيمة "Hello" نفسها لم تتغير، لكن x أصبح يشير إلى String جديد.

طيب ليه الـ String Immutable ؟

من أهم الأسباب:

- تحسين الأمان والـ Reliability.
- السماح بمشاركة الـ Strings بدون الخوف من تغييرها.
- دعم String Interning.
- جعل التعامل مع الـ Strings أكثر قابلية للتوقع.

لكن ده ممكن يكون له تأثير على الـ Performance لو بنعمل تعديلات كثيرة على الـ String.

في الحالة دي ممكن نستخدم StringBuilder لأنه يسمح بتعديل النص بدون إنشاء String جديد مع كل عملية.

```
StringBuilder sb = new StringBuilder("Hello");
```

```
sb.Append(" World");
```

الخلاصة:

String Immutability معناها إن قيمة الـ String لا تتغير بعد إنشائها.

أي تعديل على String ينتج عنه String جديد.

وفهم النقطة دي بيساعد في فهم الـ Memory Management والـ Performance في C# بشكل أفضل.

#CSharp #DotNet #String #Programming #MemoryManagement #SoftwareDevelopment #DEPI #LearningJourney

What is an Enum & When is it used?

An Enum (Enumeration) is a value type in C# used to define a set of named integer constants. It is used when a variable can only take one of a small, fixed set of possible values, improving code readability, maintainability, and type safety.

Three Common Built-in Enums in .NET:

- ConsoleColor (System): Specifies colors for the console foreground and background.
- DayOfWeek (System): Represents the days of the week.
- FileAccess (System.IO): Defines constants for Read, Write, or ReadWrite access to a file.

- When to use string:
- Small or fixed number of string operations: When performing minimal concatenation or simple string formatting.
- Immutability and Thread Safety: When you need read-only string instances that are inherently thread-safe.
- Searching and Logical Comparisons: When using built-in string functions like .Contains(), .Substring(), .StartsWith(), or performing equality comparisons.
- String Literals: For declaring fixed text constants, keys, or paths in your application.

When to use StringBuilder:

- High Performance & Memory Efficiency: When you want to avoid creating intermediate Garbage Collection (GC) pressure caused by multiple string objects in the Heap.
- Frequent Modifications: When building complex strings dynamically using multiple .Append(), .Replace(), or .Insert() operations.

سيف 1

ميكانيزم Composite Formatting باستخدام string.Format

```
string Msg = string.Format("res : {0} + {1} = {2}", X, Y, (X + Y));
```

الميكانيزم هنا يعتمد على StringBuilder داخلي تحت المظلة:

الميثود بتستقبل الـ "res : {0} + {1} = {2}" Composite Template وتمرر المتغيرات كـ Arguments.

بدل ما تجمع نصين بنصين وتكربت أوبيجكتس مؤقتة، الميثود بتروح تحسب وتعمل Parsing للقيم X و Y و (X + Y) وتحولهم لـ Strings.

بتجمع النصوص دي كلها مرة واحدة جوه Buffer داخلي بيتبني فيه النص النهائي.

أول ما يخلص تجميع النص كاملاً، بيعمل Allocate لأوبيجكت واحد بس في الـ Heap وهو الناتج النهائي المنسق.

سيف 2

الـ Interning التلقائي بيحصل للـ String Literals فقط. أما الـ Custom Interning بيستخدم للتحكم في النصوص المتكررة وقت الـ Runtime عن طريق string.Intern() لإضافتها يدوياً للـ Intern Pool أو string.IsInterned() للبحث عنها، بهدف تقليل استهلاك الـ Heap.

سيف 3

rdata Section & Jump Table.

* الـ Jump Table هو جدول بيحتوي على عناوين الحالات ويخزن في قسم القراءة فقط. rdata بالذاكرة.

* switch تنفيذ في وقت ثابت O(1) بالقفز للكود المطلوب بدلاً من التكرار بـ if-else O(N).

#Switch Evolution in C

* بدأت كأداة لمقارنة القيم الثابتة بـ (Integers, Strings)، بعدها اتطورت لتدعم الـ Pattern Matching والـ Types من C# 7.0.

* أضاف الـ Switch Expressions من C# 8.0 لتوفير كود مختصر يُرجع قيمة مباشرة دون الحاجة لـ case و break.

#Where Upperhand to C

* يختار الـ Compiler الميكانيزم الأسرع تلقائياً (Jump Table أو Hash Dictionary أو Branching) حسب نوع وعدد البيانات.

* توفر الـ C# ميزة الـ Exhaustive Checking لتتبيه المبرمج وقت الـ Build لو نسي معالجة إحدى الحالات الممكنة.

A user-defined constructor is just a constructor I write myself in the class instead of letting C# use the default one.

Its main job when creating an object is:

- Setting custom values right away: It lets me pass initial data when I create the object, so I don't get stuck with default values like null or zero.
- Making required fields mandatory: It forces anyone creating the object to provide the essential data it needs to work properly.
- Running startup code: It executes any setup logic or validation right at the moment the object is instantiated.

- **Array:** يخزن العناصر في أماكن متجاورة في الذاكرة ميزته الأساسية هي الوصول السريع لأي عنصر عن طريق الـ **Index** في وقت ثابت $O(1)$ ، لكن عيبه أن حجمه ثابت (Fixed Size) وتكلفة الإضافة أو الحذف فيه مكلفة $O(N)$ لأنها تتطلب إزاحة (Shift) باقي العناصر.
 - **Linked List:** تخزن العناصر في أماكن متفرقة في الذاكرة، وكل عنصر (Node) يحتوي على القيمة ومؤشر (Pointer) يدل على مكان العنصر التالي. ميزتها أن حجمها ديناميكي وسهلة الإضافة والحذف $O(1)$ لو المكان معروف، لكن عيبها أن الوصول لأي عنصر يتطلب المرور بالترتيب من البداية $O(N)$ ، وتستهلك ذاكرة أكبر بسبب تخزين الـ Pointers.
-

Problem 1

using System;

class Program

```
{
    static void Main()
    {
        Console.WriteLine("Enter a number: ");

        string input = Console.ReadLine();

        try
        {
            int parsedNumber = int.Parse(input);

            int convertedNumber = Convert.ToInt32(input);
        }
        catch (FormatException)
        {
            Console.WriteLine("Error: The input is not a valid integer format.");
        }
        catch (ArgumentNullException)
        {
            Console.WriteLine("Error: Input cannot be null.");
        }
    }
}
```

Difference between int.Parse and Convert.ToInt32 when handling null inputs:

- int.Parse: Throws an ArgumentNullException when the input is null.
 - Convert.ToInt32: Handles null gracefully without throwing an exception and returns 0 as the default integer value.
-

2

using System;

class Program

```
{
    static void Main()
    {
```

```

Console.Write("Please enter a number: ");

string input = Console.ReadLine();

if (int.TryParse(input, out int result))
{
    Console.WriteLine("Valid integer entered: " + result);
}
else
{
    Console.WriteLine("Error: Invalid input. Please enter a valid integer.");
}
}
}

```

Performance Efficiency, handles unexpected input from users

```

3
using System;

class Program
{
    static void Main()
    {
        object obj;

        obj = 10;

        Console.WriteLine( obj.GetHashCode());

        obj = "Hello ";

        Console.WriteLine( obj.GetHashCode());

        obj = 99.99;

        Console.WriteLine(obj.GetHashCode());
    }
}

```

بمثال بيه الارجيكت في الميموري int

```

4
using System;

class Person
{
    public string Name;
}

```

```

class Program
{
    Public static void Main()
    {
        Person p1 = new Person();

        p1.Name = "Sara";

        Person p2 = p1;

        p1.Name = "sama";

        Console.WriteLine(p2.Name);
    }
}

```

It determines whether two reference variables point to the exact same memory address in the Heap, rather than checking if their underlying values are equal.

```

5
using System;

class Program
{
    static void Main()
    {
        string text = "Hello";

        Console.WriteLine(text.GetHashCode());

        text += " Hi Willy";

        Console.WriteLine(text.GetHashCode());
    }
}

```

cannot change old value يُعمل اوبجيت جديد بالقيمة الجديد

```

6
using System;

class Program
{
    Public static void Main()
    {
        StringBuilder sb = new StringBuilder("Hi");

        Console.WriteLine( sb.GetHashCode());
    }
}

```

```

        sb.Append(" Willy");

        Console.WriteLine(sb.GetHashCode());
    }
}

```

ببعض التغيير علي نفس المكان بدل ما يعمل اوبيجكت جديد ويشغل عليه

7

ببعض التغيير في مساحة حجزها قبل كذا من غير ما يحجز مكان جديد كل مرة، ما بيعملش إهلاك للذاكرة بأوبيجكتس مؤقتة ملهاش لازمة، فيوفر وقت مسح الذاكرة.

8

```

using System;

class Program
{
    Public static void Main()
    {
        Console.Write("Enter first number: ");

        int input1 = int.Parse(Console.ReadLine());

        Console.Write("Enter second number: ");

        int input2 = int.Parse(Console.ReadLine());

        int sum = input1 + input2;

        Console.WriteLine("Sum is " + input1 + "+" + input2 + " = " + sum);

        Console.WriteLine(string.Format("Sum is {0}+{1} = {2}", input1, input2, sum));

        Console.WriteLine($"Sum is {input1}+{input2} = {sum}");
    }
}

```

الافضل لانها بتشتغل علي القيم نفسها علي طول (\$) String interpolation

9

```

using System;

class Program
{
    static void Main()
    {
        StringBuilder sb = new StringBuilder("Hello");
    }
}

```

```
sb.Append(" sara");

sb.Replace("sara", "sama");

sb.Insert(0, "aaaa ");

sb.Remove(0, 4);

}

}
```

الـ String: مصمم كـ Immutable (غير قابل للتعديل). أي عملية إضافة أو تعديل بتجبر البرنامج إنه يحجز مكان جديد تماماً في الـ Heap وينسخ النص القديم عليه، وده بيبطئ الأداء ويستهلك الذاكرة لو التعديلات كثيرة.

الـ StringBuilder: مصمم كـ Mutable (قابل للتعديل). بيحجز مساحة ذاكرة داخلية مسبقاً، ولما تعملي تعديل بيتنفذ جوة نفس المكان من غير حجز مكان جديد.
